

LSM Week 2 - R assignment

```
# W2Q1a
"
 $\chi^2$  distribution
(a) For each  $i = 1, 2, \dots, 100$  simulate 1000 random values from
Normal distribution with mean 10 and variance 16
by using command rnorm in R. Denote this data for each  $i$  as  $X_i$ 
"

n_simulations <- 100
n_values <- 1000
pop_mean <- 10
pop_variance <- 16
pop_sd <- sqrt(pop_variance)

X_i <- vector("list", n_simulations)

for (i in 1:n_simulations) {
  X_i[[i]] <- rnorm(n_values, mean = pop_mean, sd = pop_sd)
}

print(X_i)
```

```

[73] 14.82411383  8.25692656  6.08850568  2.91158946 16.02407319 12.09869018  9.11633706  6.08796533
[81] 11.28838363  4.25851318  0.22314029 11.41049983 12.05960587  9.35495868 -1.00585766 16.79101461
[89] 9.51592011 10.74794359 10.53154034 17.55671108 7.28080434 11.39285935  8.74642893  6.68521150
[97] 10.82409196  9.89353215 13.13932174  2.59614312 17.86895600 14.02939169  9.85833455  7.04059029
[105] 15.60699291  5.22110448  6.48919156  9.68574809 14.84190931  4.69008264  1.91814675 12.13751086
[113] 11.20804325  9.91288603  8.87405612  6.14373087  9.17289521  4.61148897 15.15695234 10.38130461
[121] 8.24922542 11.27254520 10.87178053 14.41613367  6.50474726 12.97119210  9.34904428  7.71376360
[129] 14.80558651 10.72760995  9.27374166 16.96130291 11.51414330 17.10502764  9.58962270 18.31779132
[137] 10.86678902  4.25773427  7.86064526  8.57996919 14.37538056  7.44050514 15.10041388  3.50730642
[145] 13.08957596 12.96699723  9.67973814  9.35407364  9.09620586 12.85977563 12.63188815 13.24844691
[153] 16.36536046  6.20338464 10.76451479  5.54376856 10.59687043  6.34191049  5.05636576 13.73578948
[161] 3.82553728 11.57322078  5.33909280 10.61051088  8.46966599  9.10321623  5.18174531 13.07237802
[169] 3.92705815  4.38018839  8.39837033  8.36612930 11.71751716 10.73838137 14.88637991  9.75751332
[177] 12.08940953 10.92312955  3.92660029 19.06272670 12.89497318 12.88491186  4.56513507  9.35430749
[185] 11.05453588 10.53091298  9.66080588 15.19420006 17.65124142 14.83194131 17.02436504  1.11534611

```

```
# W2Q1b
"
Calculate the means of each  $X_i$  ( $i = 1, 2, \dots, 100$ ) and
store it in a new vector 'Mi'.
"
M_i <- sapply(X_i, mean)
print(M_i)
```

```
r># M_i <- sapply(X_i, mean)
print(M_i)
```

[1]	0.065705	9.991259	0.023431	0.035240	0.126861	0.132325	0.073897	9.901985
[9]	0.322074	0.003534	9.961021	0.026193	9.958126	9.996328	0.099817	10.109798
[17]	0.054186	0.008927	0.016117	9.805999	0.064239	9.980406	0.112630	0.028741
[25]	10.109396	9.892281	0.207334	9.989938	9.771835	9.775668	9.867574	0.016297
[33]	0.030871	0.007272	0.183559	0.216354	0.038344	9.980352	0.109682	9.845816
[41]	9.930394	0.172650	0.028353	0.178623	9.991241	0.073486	0.016633	0.038573
[49]	9.918890	9.938990	0.028984	9.943369	0.084756	0.114154	0.079191	0.178756
[57]	9.986554	0.074039	0.212607	0.015782	0.025442	9.891981	9.982898	9.840442
[65]	9.862317	0.054694	9.931392	9.858991	0.055237	0.106360	0.251511	0.080507
[73]	10.196121	0.144373	9.993956	9.911279	0.032755	9.825809	0.051394	0.172436
[81]	9.982499	0.046621	0.260296	9.984999	0.049128	9.994537	0.088156	0.055942
[89]	9.883046	9.863958	0.047165	0.064965	9.994265	9.948732	0.230730	9.942863
[97]	0.192947	9.872478	9.952274	0.089014				

```
# W2Q1c
```

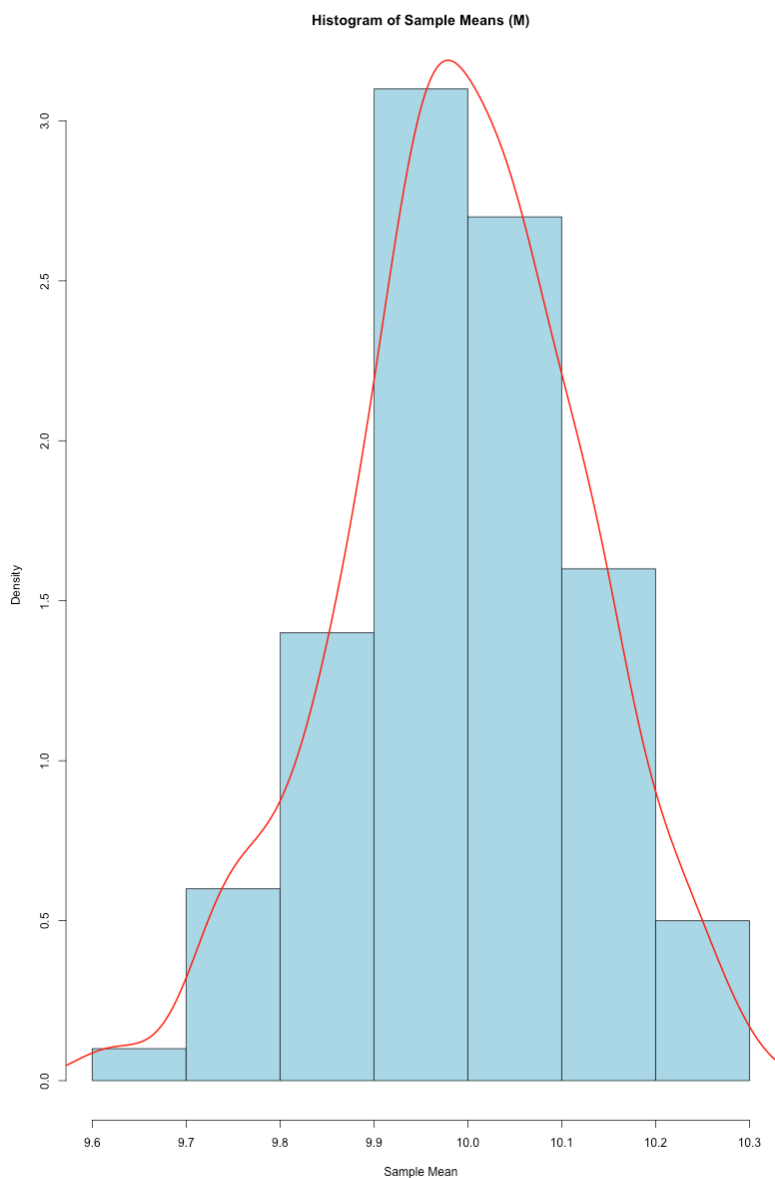
```
"
```

Plot the histogram of 'M = {M_i: 1 ≤ i ≤ 100}' using the command hist() in R-software. What should the distribution of M be?

```
"
```

```
hist(M_i,  
     main = "Histogram of Sample Means (M)",  
     xlab = "Sample Mean",  
     col = "lightblue",  
     border = "black",  
     probability = TRUE  
)  
lines(density(M_i), col = "red", lwd = 2) # Add a density curve
```

According to the Central Limit Theorem (CLT), the distribution of sample means
(M_i) should be approximately Normal. Since the underlying population is Normal,
the distribution of sample means is exactly Normal.
The theoretical mean of this distribution is the population mean, $\mu = 10$.
The theoretical variance is $\sigma^2/n = 16/1000 = 0.016$.
So, M_i should follow $N(10, 0.016)$.



```
# W2Q1d
```

```
"
```

```
Calculate the sample variance  $S_i^2$   
of each  $X_i$  ( $i = 1, 2, \dots, 100$ ).
```

```
"
```

```
S_sq <- sapply(X_i, var)  
print(S_sq)
```

```
r$> S_sq <- sapply(X_i, var)  
print(S_sq)  
[1] 16.90920 15.62241 17.14593 15.62770 16.69176 17.09954 15.12540 16.51528 16.82294  
[10] 16.28364 13.79157 15.60124 16.94265 16.18191 15.11927 16.55743 15.58905 15.56026  
[19] 15.76738 15.21276 15.13983 16.60040 16.55060 15.09485 15.00326 16.44210 15.97021  
[28] 15.66829 17.35892 16.75111 17.20010 15.33302 14.76591 17.08436 16.57601 18.02516  
[37] 16.56356 15.88443 17.66576 16.81292 16.16297 15.21721 15.19950 15.70410 16.59231  
[46] 16.21687 16.89151 16.31109 16.36979 16.22981 15.96513 14.51330 14.46617 16.60209  
[55] 16.72923 16.97574 15.78869 16.53671 15.55022 16.82045 16.02922 16.06774 16.51702  
[64] 14.81816 16.41051 15.23513 15.81840 15.42609 17.14925 15.02600 17.91502 16.23071  
[73] 16.24942 15.76215 16.33513 15.87391 15.49228 17.23697 15.95165 15.60956 15.95811  
[82] 16.79445 16.13715 16.16382 16.70537 17.72153 16.33811 16.20809 16.67580 15.38690  
[91] 14.57607 16.47610 14.94702 16.96061 17.21666 16.45453 16.77203 15.84878 15.72334  
[100] 15.10365
```

```
# W2Q1e
```

```
"
```

```
Plot the histogram of 'CHISQ= {(999/16) $S_i^2$ :  $1 \leq i \leq 100$ }'  
using the command hist() in R-software.
```

```
What should the distribution of CHISQ be?
```

```
"
```

```
CHISQ <- (n_values - 1) / pop_variance * S_sq
```

```
hist(CHISQ,  
  main = "Histogram of the Chi-Square Statistic",  
  xlab = "Value of (n-1) $S^2$  /  $\sigma^2$ ",  
  col = "lightgreen",  
  border = "black",  
  probability = TRUE,  
  breaks = 20  
)
```

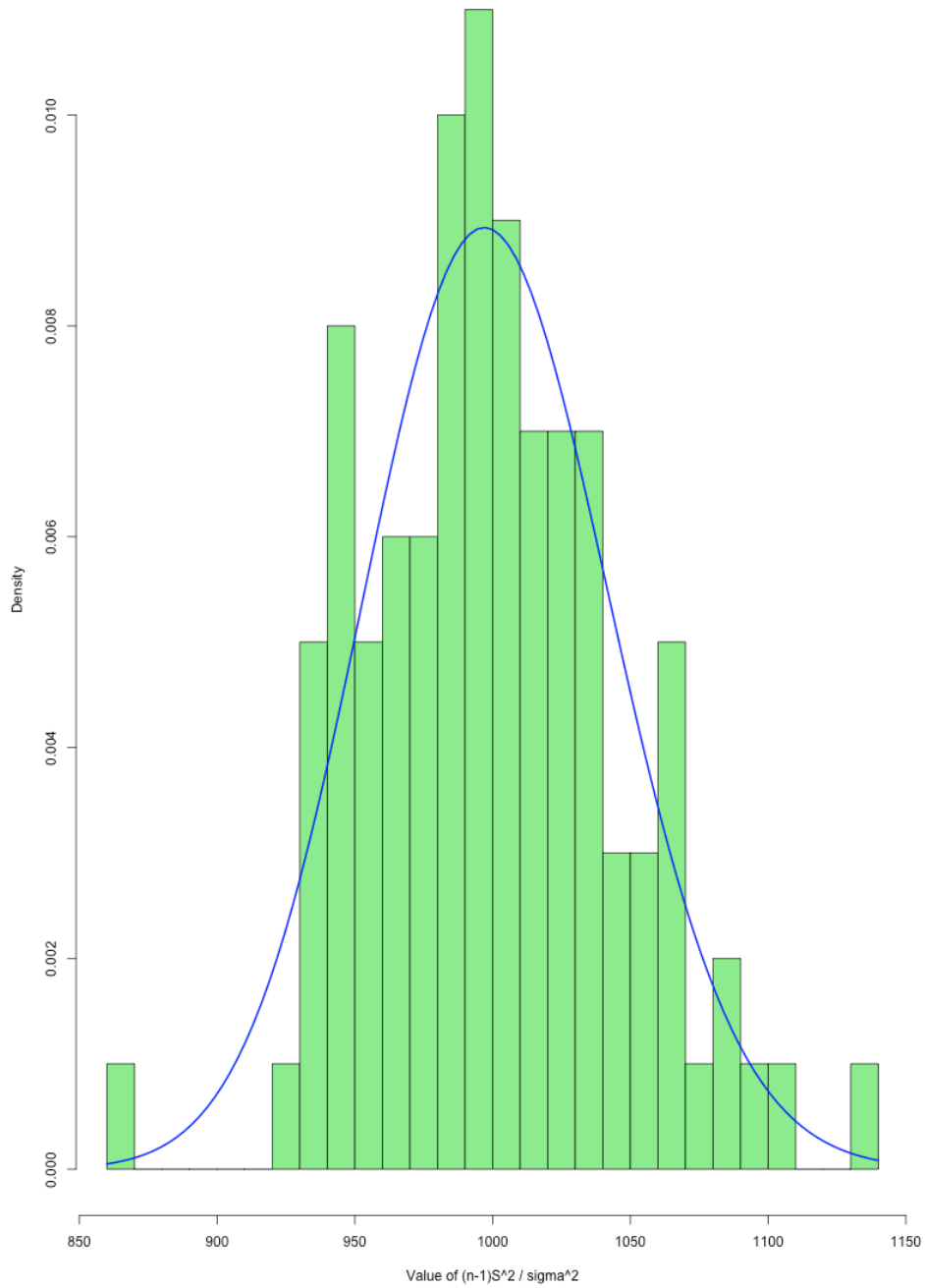
```
# Overlay the theoretical chi-square density curve
```

```
curve(dchisq(x, df = n_values - 1), add = TRUE, col = "blue", lwd = 2)
```

```
# the CHISQ vector should follow a Chi-squared
```

```
# distribution with 999 degrees of freedom.
```

Histogram of the Chi-Square Statistic



```
# W2Q1f
```

```
"
```

```
Calculate the statistic  $T_i = (M_i - 10)/(S_i/\sqrt{1000})$ 
```

```
for each  $i = 1, 2, \dots, 100$ .
```

```
"
```

```
S <- sqrt(S_sq)
```

```
T_i <- (M_i - pop_mean) / (S / sqrt(n_values))
```

```
print(T_i)
```

```

r$> S <- sqrt(S_sq)
T_i <- (M_i - pop_mean) / (S / sqrt(n_values))
print(T_i)
[1] 0.505288189 -0.069936169 0.178938298 0.281896230 0.981919958 1.011931352
[7] 0.600863721 -0.762694450 2.483161445 0.027697747 -0.331916080 0.209705958
[13] -0.321704553 -0.028867823 0.811784811 0.853295130 0.433986960 0.071563485
[19] 0.080566711 -1.572969023 0.522077405 -0.152074985 0.875480066 0.233934463
[25] 0.893115460 -0.840063528 1.640650681 -0.008487572 -1.731757287 -1.733283887
[31] -1.009735349 0.131615395 0.313303135 0.055636221 1.425720786 1.611483382
[37] 0.297936323 -0.155891481 0.148079266 -1.189100026 -0.547500298 1.399582393
[43] 0.229975764 -2.245341754 -0.068000063 0.577056191 0.127978678 0.302021628
[49] -0.633949387 -0.478902521 0.229389576 -0.470082765 0.704681387 0.885954191
[55] 0.612260873 1.371975300 -0.107009918 0.575754885 1.704940064 0.121688097
[61] 0.200953919 -0.852165973 -0.133073844 -1.310755486 -1.074778258 0.443112369
[67] -0.689410380 -1.135325416 0.421800064 0.867671844 1.879096038 0.631922945
[73] 1.538527614 1.149945576 -0.474537882 -0.704181037 0.263163325 -1.326770526
[79] 0.406921692 1.380170214 -0.134976089 0.359751651 2.049055690 -0.117987572
[85] 0.380104850 -0.041041123 0.689685172 0.439415672 -0.905672476 -1.096728133
[91] 0.390664409 0.506119842 -0.046910837 -0.393663666 1.758448860 -0.445421359
[97] 0.997996333 -1.012951715 -0.380529738 0.724294285

```

W2W1g

"

Plot the histogram of 'T = {Ti: i = 1, 2, 3, . . . , 100}'

using the command hist() in R-software.

What should the distribution of T be?

"

hist(T_i,

main = "Histogram of the T-Statistic",

xlab = "T-statistic Value",

col = "salmon",

border = "black",

probability = TRUE,

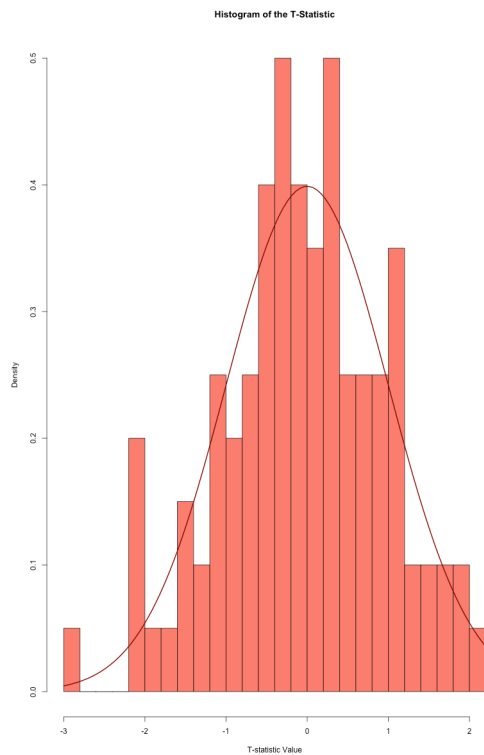
breaks = 20

)

Overlay the theoretical t-distribution curve

curve(dt(x, df = n_values - 1), add = TRUE, col = "darkred", lwd = 2)

T should follow a t-distribution with 999 degrees of freedom.



```
# W2Q2a
```

```
"
```

Give a brief description of the dataset and
read the data set as data frame (name it 'df') in R.

```
"
```

```
data("faithful")
```

```
str(faithful)
```

```
df <- faithful
```

```
r$> # W2Q2a
"
  Give a brief description of the dataset and
  read the data set as data frame (name it 'df') in R.
"
data("faithful")
str(faithful)

df <- faithful
[1] "\nGive a brief description of the dataset and\nread the data set as data frame (name it 'df') in R.\n"
'data.frame':  272 obs. of  2 variables:
 $ eruptions: num  3.6 1.8 3.33 2.28 4.53 ...
 $ waiting  : num  79 54 74 62 85 55 88 85 51 85 ...
```

```
# W2Q2b
```

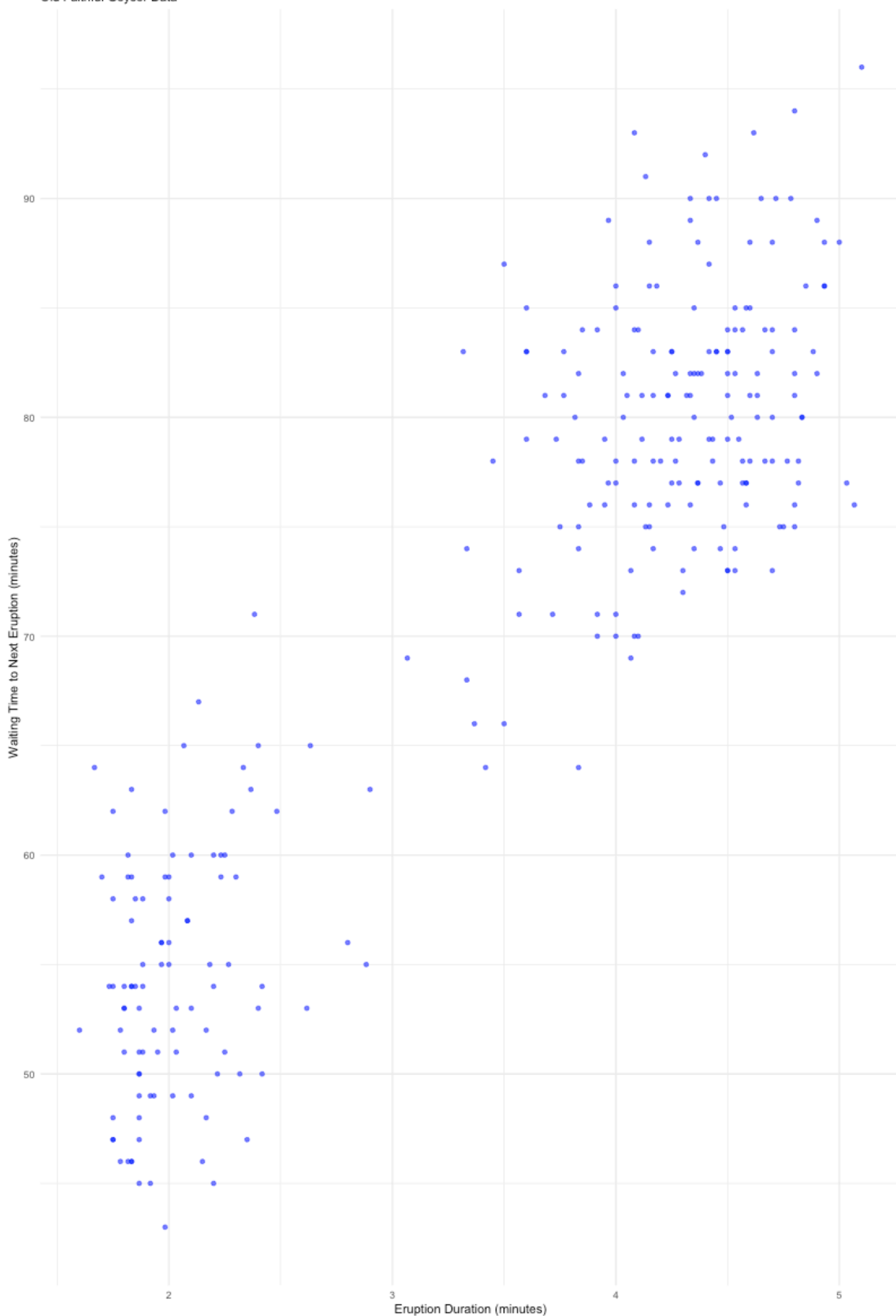
```
"
```

```
Using 'ggplot' in R, plot the scatter plot between the variables  
'waiting' and 'eruptions'. On visualizing the scatter-plot, c  
omment on the relationship between 'waiting' and 'eruptions'.  
"
```

```
ggplot(df, aes(x = eruptions, y = waiting)) +  
  geom_point(alpha = 0.6, color = "blue") +  
  labs(  
    title = "Scatter Plot of Waiting Time vs. Eruption Duration",  
    subtitle = "Old Faithful Geyser Data",  
    x = "Eruption Duration (minutes)",  
    y = "Waiting Time to Next Eruption (minutes)"  
  ) +  
  theme_minimal()
```

```
# The scatter plot shows a positive linear relationship between the duration of  
# an eruption and the waiting time until the next eruption. This means that  
# longer eruptions tend to be followed by longer waiting periods. The data also  
# appears to be clustered into two distinct groups, suggesting that a simple  
# linear model might not capture the full complexity of the relationship.
```

Scatter Plot of Waiting Time vs. Eruption Duration
Old Faithful Geyser Data




```
# W2Q2c
```

```
# First, create the base scatter plot
```

```
p_base <- ggplot(df, aes(x = eruptions, y = waiting)) +  
  geom_point(alpha = 0.4, color = "black") +  
  labs(  
    title = "200 Random Linear Models on the Faithful Dataset",  
    x = "Eruption Duration (minutes)",  
    y = "Waiting Time to Next Eruption (minutes)"  
  )
```

```
# Initialize a data frame to store model parameters and their MSE
```

```
random_models <- data.frame(  
  intercept = numeric(200),  
  slope = numeric(200),  
  mse = numeric(200)  
)
```

```
# Generate 200 random models
```

```
for (i in 1:200) {  
  # Generate random intercept and slope from a plausible range  
  # Based on the plot, intercept is ~30-40, slope is ~10  
  b0 <- runif(1, min = 20, max = 50)  
  b1 <- runif(1, min = 5, max = 15)
```

```
  # Calculate predicted values and Mean Squared Error (MSE)
```

```
  predictions <- b0 + b1 * df$eruptions  
  mse <- mean((df$waiting - predictions)^2)
```

```
  # Store the results
```

```
  random_models[i, ] <- c(b0, b1, mse)
```

```
  # Add the line to the plot
```

```
  p_base <- p_base + geom_abline(intercept = b0, slope = b1, color = "gray", alpha = 0.5)  
}
```

```
# Print the plot with all 200 random lines
```

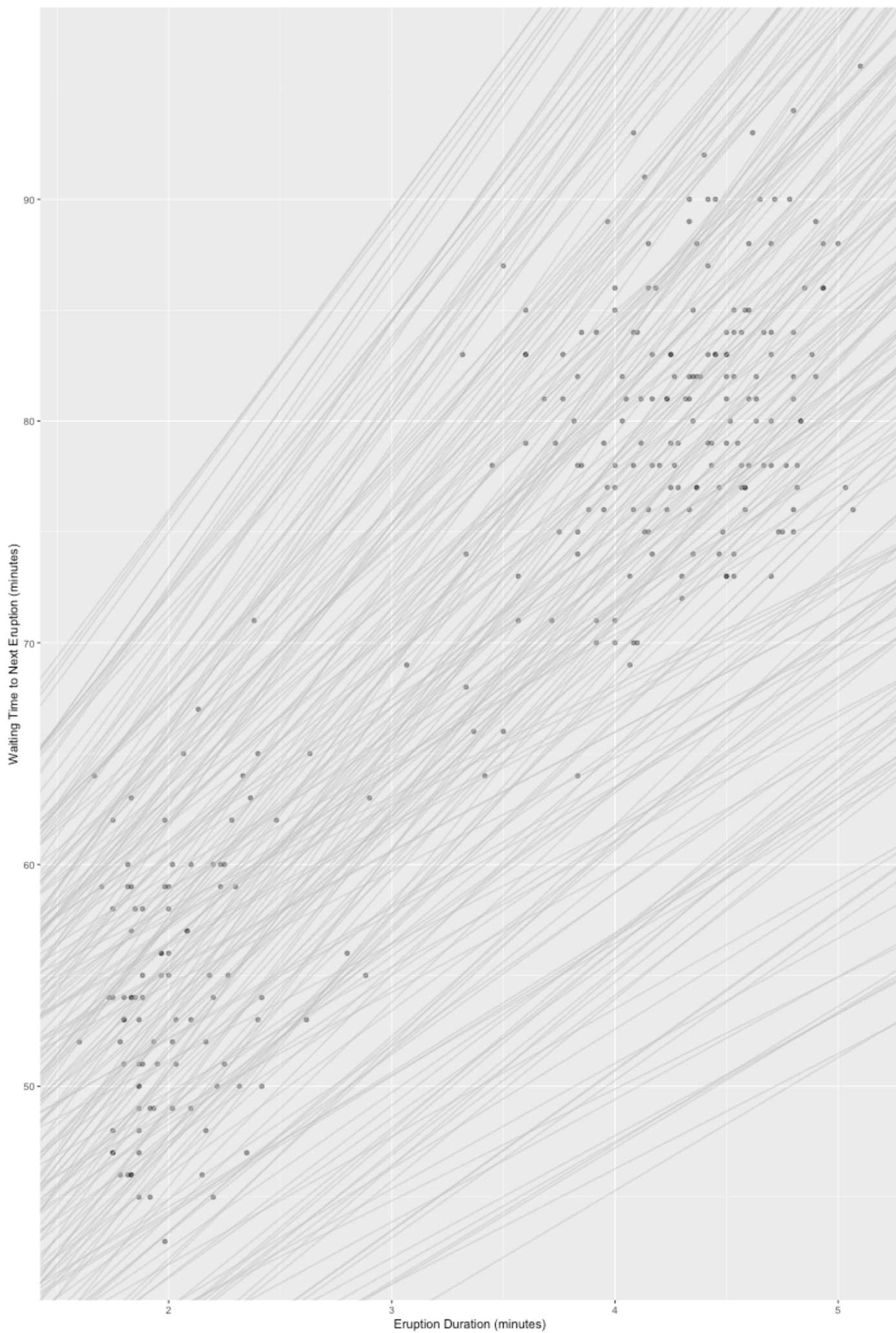
```
print(p_base)
```

```
# The `random_models` dataframe now contains the MSE for each model
```

```
head(random_models)
```

```
}  
  
# Print the plot with all 200 random lines  
print(p_base)  
  
# The `random_models` dataframe now contains the MSE for each model  
head(random_models)  
intercept    slope      mse  
1 31.09337  6.905745 300.75088  
2 47.92091  6.509671  57.90611  
3 44.93441  6.429117  71.24960  
4 40.69733  5.239675 216.04052  
5 31.69843  6.256057 362.71869  
6 24.57575 11.600908  70.04148
```

200 Random Linear Models on the Faithful Dataset

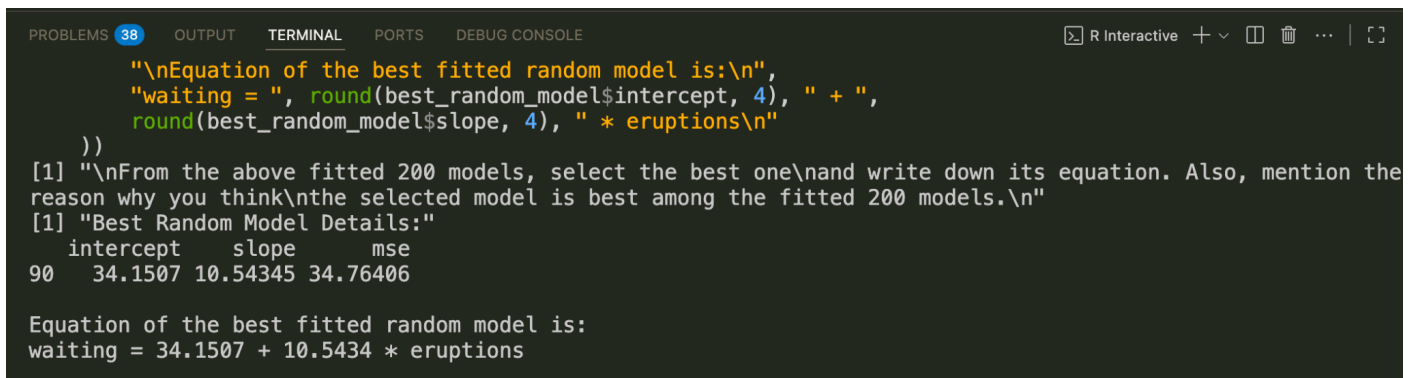


```
# W2Q2d
"
From the above fitted 200 models, select the best one
and write down its equation. Also, mention the reason why you think
the selected model is best among the fitted 200 models.
"

# The best model is the one that minimizes the Mean Squared Error (MSE).
best_random_model <- random_models[which.min(random_models$mse), ]

# Print the details of the best model
print("Best Random Model Details:")
print(best_random_model)

# Write down its equation
cat(paste0(
  "\nEquation of the best fitted random model is:\n",
  "waiting = ", round(best_random_model$intercept, 4), " + ",
  round(best_random_model$slope, 4), " * eruptions\n"
))
```



The screenshot shows an R Interactive terminal window with the following content:

```

PROBLEMS 38 OUTPUT TERMINAL PORTS DEBUG CONSOLE
"\nEquation of the best fitted random model is:\n",
"waiting = ", round(best_random_model$intercept, 4), " + ",
round(best_random_model$slope, 4), " * eruptions\n"
))
[1] "\nFrom the above fitted 200 models, select the best one\nand write down its equation. Also, mention the
reason why you think\nthe selected model is best among the fitted 200 models.\n"
[1] "Best Random Model Details:"
      intercept      slope      mse
90    34.1507 10.54345 34.76406

Equation of the best fitted random model is:
waiting = 34.1507 + 10.5434 * eruptions

```

```
# W2Q2e
"
Use the inbuilt function lm() in R, to find the equation of
best fitted linear model ('waiting' on 'eruptions') for the 'faithful' dataset.
"

model_lm <- lm(waiting ~ eruptions, data = df)

# Get the coefficients
lm_coefs <- coef(model_lm)

# Print the coefficients
print("Coefficients from the lm() function:")
print(lm_coefs)

# Write the equation of the best-fitted model from lm()
cat(paste0(
  "\nEquation of the best fitted model using lm() is:\n",
  "waiting = ", round(lm_coefs[1], 4), " + ",
```

```
round(lm_coefs[2], 4), " * eruptions\n"
))
```

The screenshot shows an R Interactive terminal window with the following content:

```
round(lm_coefs[2], 4), " * eruptions\n"
))
```

Output:

```
[1] "\nUse the inbuilt function lm() in R, to find the equation of\nbest fitted linear model ('waiting' on 'e
rptions') for the 'faithful' dataset.\n"
[1] "Coefficients from the lm() function:"
(Intercept)    eruptions
  33.47440      10.72964
```

Equation of the best fitted model using lm() is:
waiting = 33.4744 + 10.7296 * eruptions

```
# W2Q2f
```

```
"
```

Using 'ggplot' in R, plot the linear models obtained in part (d) and (e) in the same scatter plot (plotted in part (b)).

```
"
```

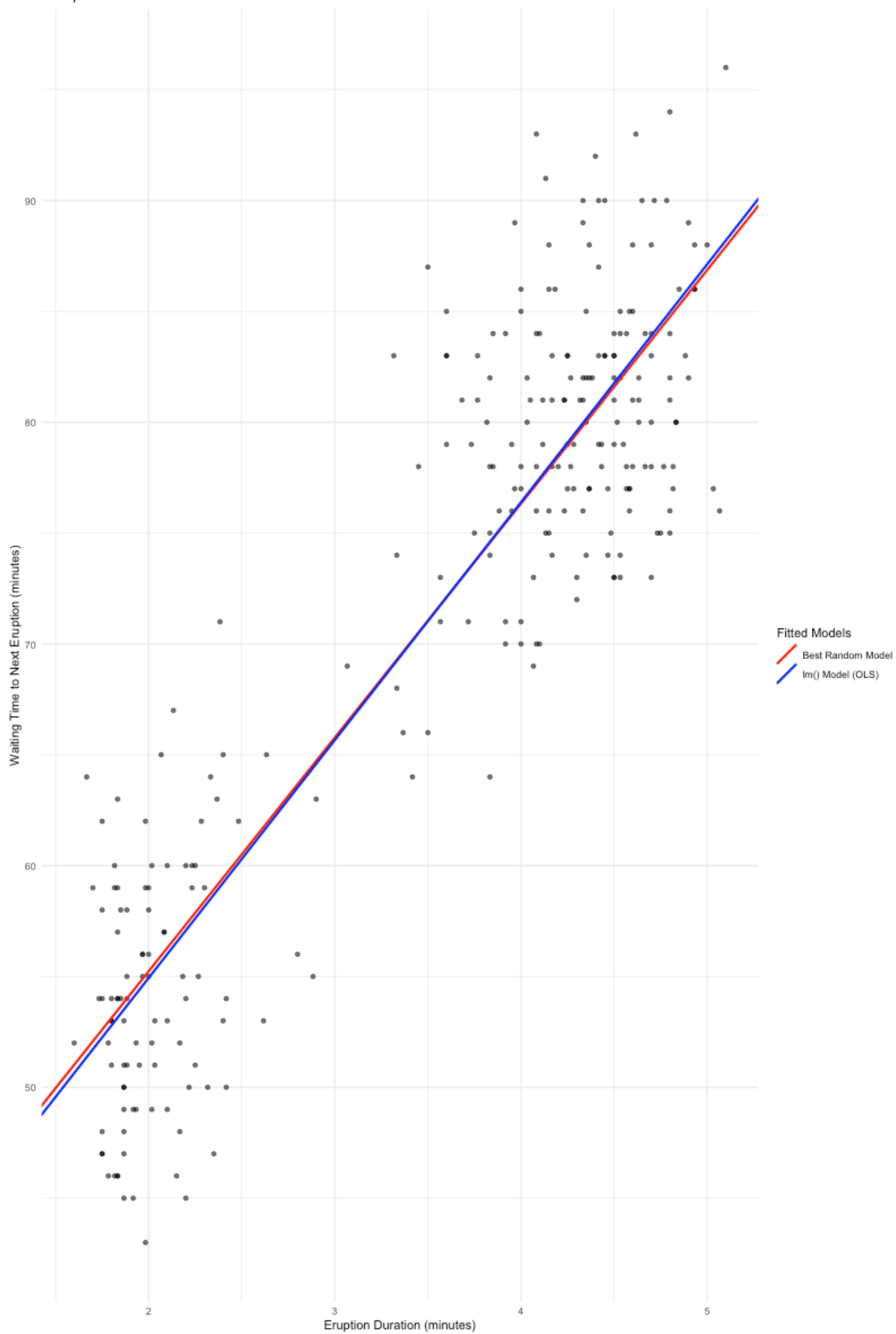
```
# Create a data frame containing the parameters for both lines
```

```
line_models <- data.frame(
  model_type = c("Best Random Model", "lm() Model (OLS)"),
  intercept = c(best_random_model$intercept, lm_coefs[1]),
  slope = c(best_random_model$slope, lm_coefs[2])
)
```

```
# Now, create the plot using this new data frame for the ablines
```

```
ggplot(df, aes(x = eruptions, y = waiting)) +
  geom_point(alpha = 0.6, color = "black") +
  labs(
    title = "Comparison of Best Random Model and OLS Model",
    x = "Eruption Duration (minutes)",
    y = "Waiting Time to Next Eruption (minutes)"
  ) +
  # Use the new data frame and map aesthetics for intercept, slope, and color
  geom_abline(
    data = line_models,
    aes(intercept = intercept, slope = slope, color = model_type),
    linewidth = 1
  ) +
  # The manual scale now works perfectly with the 'model_type' variable
  scale_color_manual(
    name = "Fitted Models",
    values = c("Best Random Model" = "red", "lm() Model (OLS)" = "blue")
  ) +
  theme_minimal()
```

Comparison of Best Random Model and OLS Model



```
# W2Q2g
```

```
"
```

For each of the fitted models (part (d) and (e)),
plot the 'residuals' vs 'eruptions' using 'ggplot' in R.
Comment on the obtained plots.

```
"
```

```
# 1. Residual plot for the best random model from part (d)
```

```
df$residuals_random <- df$waiting - (best_random_model$intercept + best_random_model$slope *  
df$eruptions)
```

```
plot_random_residuals <- ggplot(df, aes(x = eruptions, y = residuals_random)) +  
  geom_point(alpha = 0.6) +  
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +  
  labs(  
    title = "Residuals vs. Eruptions (Best Random Model)",  
    x = "Eruption Duration (minutes)",  
    y = "Residuals"  
  ) +  
  theme_minimal()
```

```
print(plot_random_residuals)
```

```
# 2. Residual plot for the lm() model from part (e)
```

```
df$residuals_lm <- residuals(model_lm)
```

```
plot_lm_residuals <- ggplot(df, aes(x = eruptions, y = residuals_lm)) +  
  geom_point(alpha = 0.6) +  
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +  
  labs(  
    title = "Residuals vs. Eruptions (lm() Model)",  
    x = "Eruption Duration (minutes)",  
    y = "Residuals"  
  ) +  
  theme_minimal()
```

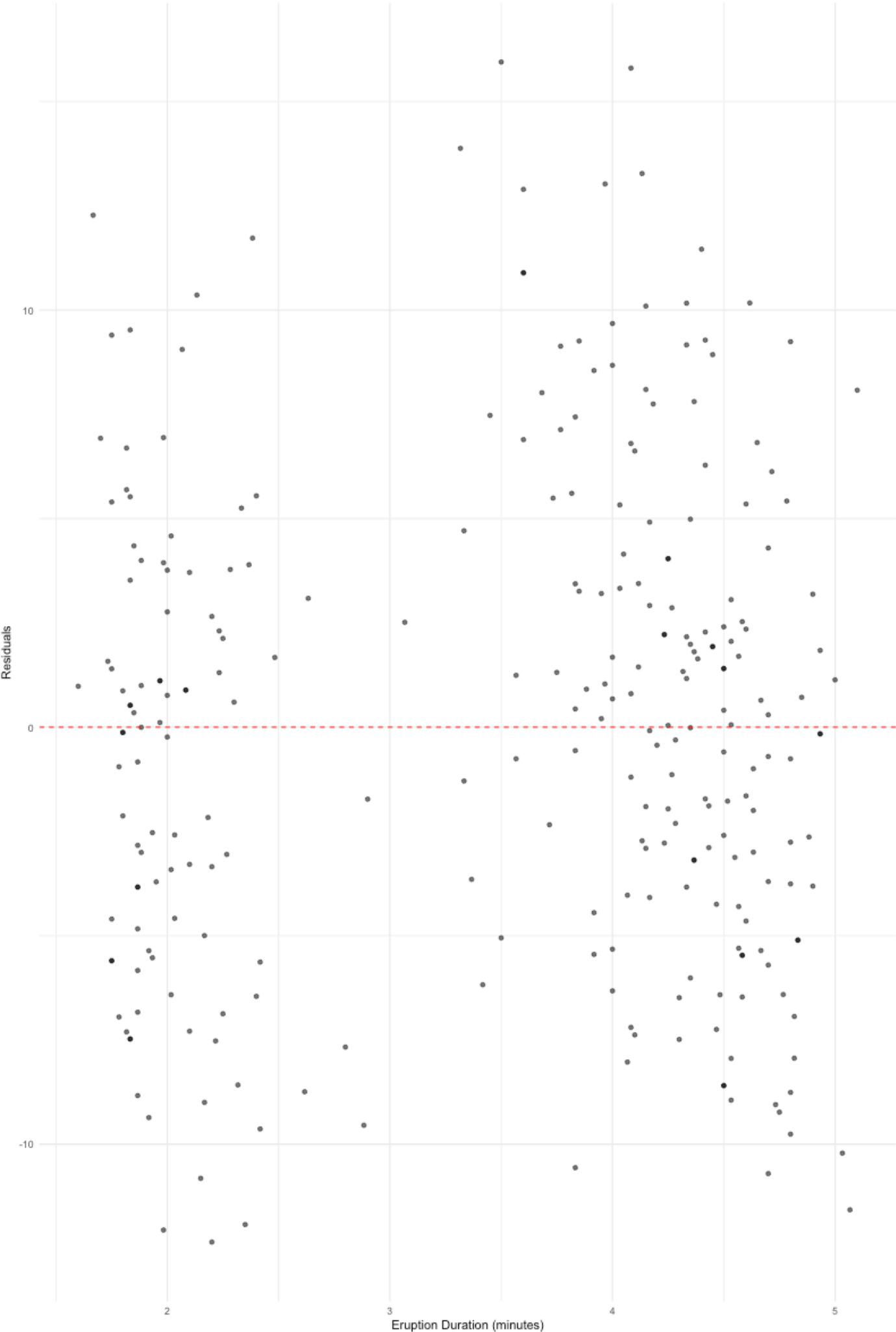
```
print(plot_lm_residuals)
```

Both residual plots — one from the Best Random Model and the other from the lm() model — display a
very similar pattern. The residuals are spread around the horizontal line at zero, but they show a distinct
clustering pattern for shorter and longer eruption durations (around 2 and 4–5 minutes respectively). This
indicates that the residuals are not randomly scattered and there is non-linearity present in the
relationship between eruption duration and waiting time.

```
# Conclusion:
```

```
# Residuals are centred around zero (no major bias).  
# Variance appears roughly constant.  
# Clear clustering pattern suggests a non-linear relationship between waiting time and eruption duration.  
# The linear assumption of lm() is partially violated.
```

Residuals vs. Eruptions (Best Random Model)



Residuals vs. Eruptions (lm() Model)

